

Customer Identity Resolution for Multi-Source Customer Data

How the current Customer 360 implementation matches, merges, and enriches profiles

Trieu Nguyen

2026-09-13

How does identity resolution merge two profiles from multi data sources ?

It does not merge two raw rows directly. It processes each raw profile in order, finds the best existing master profile within the same tenant and business domain, and then either links and consolidates the raw row into that master or creates a new master. Therefore, when two source records describe the same customer, the first record normally creates the master profile and the second record is matched to it through one or more configured identity attributes.

For example:

```
Adjust record A:      email = H, device_id = D1, source_system = Adjust
                      |
                      v
                    create master M
                    email = H, device_ids = [D1]
                      ^
                      |
Web Tracking record B: email = H, cookie_id = C1, source_system = WebTracking
                      |
                      v
                    link B -> M and consolidate
                    device_ids = [D1], cookie_ids = [C1]
```

Here H can be the same SHA-256 digest in both rows. The resolver does not reverse the digest. It uses equality of the stored values as a deterministic match, records the raw-to-master relationship, preserves the source identifiers, and leaves an auditable lineage path from the master back to both raw records.

Abstract

Customer Identity Resolution (CIR) converts heterogeneous source observations into a tenant-scoped Customer 360 master profile. The current implementation uses a PostgreSQL staging queue, a Python resolver, metadata-driven matching rules, explicit row-level tenant context, and a link table that records every raw-to-master decision. Matching is intentionally separate from downstream persona resolution: the resolver first answers whether a raw observation belongs to an existing identity, then computes an explainable, versioned persona for the resulting master profile.

This paper describes the implementation present in `database-init/database-schema.sql`, `database-init/init-core` and `backend-system/identity_resolution/identity_resolution/`. It also identifies schema capabilities that are present but not yet part of the active resolver path, so the design description does not overstate current behavior.

1. Problem and Design Answer

Multiple systems observe the same person under different identifiers. A mobile attribution platform may know a device and an external customer ID; a web tracker may know a cookie; a CRM or banking

system may know an email, phone number, or national ID. CIR must decide whether each observation belongs to an existing customer, preserve the observation, and update the unified profile without allowing data to cross tenant or domain boundaries.

The current solution follows this sequence:

1. Land every source observation in `cdp_raw_profiles_stage`.
2. Load active matching and consolidation metadata from `cdp_profile_attributes`.
3. Select new rows for one tenant at a time and set the PostgreSQL tenant context.
4. Build candidate predicates for each populated, active identity attribute.
5. Search only master profiles in the same tenant and domain.
6. Rank the matching candidates by the proportion of satisfied predicates.
7. Link the raw row to the best candidate and merge its data, or create a new master and first link.
8. Recompute the resulting profile's persona, mark the raw row processed, and commit the batch.

The main invariant is an application-level goal: one logical customer master per tenant and domain. The database strictly enforces that a raw profile has at most one link per tenant through `UNIQUE (tenant_id, raw_profile_id)`, while the master table stores the consolidated record and lineage. A master-to-master merge is a separate operation and is not performed by the current `CustomerIdentityResolver` batch.

2. Data Model

2.1 Raw staging: `cdp_raw_profiles_stage`

This table is the queue and source-lineage record for inbound observations. It includes:

- `raw_profile_id`, `tenant_id`, `user_id`, and `domain` for identity, ownership, and scope.
- `source_system` and `channel` for source and delivery context.
- Core identifiers such as `external_customer_id`, `email`, `phone_number`, `national_id`, and `name` fields.
- Address components, company information, device identifiers, advertising IDs, cookies, GA client/session IDs, and push tokens.
- Attribution fields such as `media_source`, campaign data, UTM data, and Adjust metadata.
- `event_name`, `event_time`, and `event_payload` for the original event and extensible business attributes.
- `status_code`, `processed_at`, and `created_at` for queue processing.

The schema documents `status_code` = 1 as new, 2 as in progress, 3 as processed, 0 as inactive, and -1 as delete. The current resolver selects only `status_code` = 1 and changes successfully handled rows to 3; it does not currently mark failed rows as 4.

2.2 Golden identity: `cdp_master_profiles`

This is the durable identity record. It contains:

- `master_profile_id`, `tenant_id`, and `domain` as the primary identity, scope, and business classification.
- Core profile fields such as `full_name`, `email`, `phone_number`, `address` as JSONB, `company_name`, and demographic fields.
- Consolidated identity collections: `external_ids` as a source-keyed JSONB map, `device_ids`, `advertising_ids`, and `cookie_ids` as arrays, and `push_tokens` as JSONB.
- Lineage fields: `source_systems`, `first_seen_raw_profile_id`, `linked_raw_profile_count`, and `last_identity_resolved_at`.
- Lifecycle, engagement, CLV, churn, data-quality, and identity-confidence fields populated by other pipelines or the persona engine.
- `is_hashed`, `persona_name`, `persona_summary`, and `current_persona_id` for privacy-aware display and persona linkage.

The database check constraint requires `persona_name` whenever `is_hashed` = TRUE. The resolver satisfies that constraint by generating a non-PII label when a populated PII field looks like a 64-character SHA-256 hexadecimal digest.

2.3 Domain context: `cdp_domain_profiles`

Domain-specific data no longer belongs in arbitrary scalar columns on the master row. A master can have one domain profile per `sys_domain`, enforced by `UNIQUE (master_profile_id, domain_id)`. The

domain profile stores domain_attributes JSONB, lifecycle and engagement data, persona information, analytics, and activity timestamps.

This is where values such as banking national_id, kyc_status, cif_number, retail loyalty fields, travel loyalty data, media subscriptions, and education identifiers are stored. The resolver reads and writes domain attributes through the domain profile when a matching or consolidation rule references them.

2.4 Raw-to-master lineage: cdp_profile_links

Each link records the decision for one raw observation:

- tenant_id, raw_profile_id, and master_profile_id identify the relationship.
- match_score stores the computed score for a matched candidate; a newly created master uses a null score.
- match_method records DynamicMatch:<fields> for a candidate match or NewMaster for creation.
- status supports ACTIVE, HISTORICAL, UNLINKED, and SUPERSEDED, with un-link metadata for future split and correction workflows.

The unique (tenant_id, raw_profile_id) constraint makes retry behavior idempotent at the link level. The current resolver uses ON CONFLICT DO NOTHING when adding a link and checks for an existing link before creating a new master after a partial retry.

2.5 Metadata: cdp_profile_attributes

The attribute catalog describes where a field lives and how it participates in CIR. Relevant columns are:

- attribute_internal_code and master_profile_column for the raw attribute and consolidated destination.
- source_table, domain_scope, is_pii, and status for catalog and governance metadata.
- is_identity_resolution, matching_rule, and matching_threshold for candidate matching.
- consolidation_rule and consolidation_config for conflict resolution.
- blocked_values, blocked_patterns, value_limit, and limit_timeframe for identifier-governance metadata used by surrounding platform workflows.

The resolver loads rows where is_identity_resolution = TRUE, status = 'ACTIVE', and matching_rule is neither null nor none. The catalog contains domain information, but the resolver's rule query is global; it does not add a domain_scope predicate. Domain isolation is enforced during candidate matching and domain-attribute access, not by selecting a different ruleset for each domain.

3. End-to-End Resolution Flow

Step 1: Ingest and stage

Connectors write observations from Adjust, OneSignal, Web Tracking/GA4, POS, Core Banking, CRM, and other sources to cdp_raw_profiles_stage. The raw row remains the source of truth for the observation, including the original event payload and attribution information. A source may normalize values before insertion; the demo seeder hashes normalized PII so the same logical value from different systems produces the same digest without storing plaintext.

Step 2: Load active rules

CustomerIdentityResolver._get_active_rules() reads the active rule metadata once at the beginning of a batch. A rule has a matching family and may also have a consolidation policy. The catalog seeds exact matching for email, phone, external customer ID, device ID, advertising ID, and cookie ID, plus exact or fuzzy address components, company-name trigram matching, and banking national-ID matching through domain JSONB. In the current batch implementation, however, RAW_PROFILE_COLUMNS does not project the address or company fields, so those catalog rules are not evaluated until the resolver's raw-column projection is extended. The fields currently available to the active matching path include the core keys and national_id.

fuzzy_dmetaphone is supported by the resolver and the fuzzystmatch PostgreSQL extension, but it is not a default seeded rule in init-core-database.sql. Names are deliberately not an active default identity key: shared names are too collision-prone to establish identity by themselves.

Step 3: Establish tenant context

The batch clears the tenant context while enumerating tenant IDs, then calls `SET app.tenant_id = <tenant>` before reading or mutating rows for each tenant. PostgreSQL RLS is enabled and forced for tenant-owned CDP tables. Policies compare `tenant_id` with `current_setting('app.tenant_id', true)` and fail closed when the setting is empty.

The resolver also includes explicit `tenant_id` predicates in its key queries. This defense in depth matters because PostgreSQL superuser or `BYPASSRLS` connections can bypass RLS; production application roles should be non-superuser roles without `BYPASSRLS`.

Step 4: Select a new raw row

For each tenant, `_fetch_unprocessed_profiles()` selects an explicit list of raw columns where `tenant_id = %s` and `status_code = 1`, limited by `batch_size`. The production daily entry point bounds `CIR_BATCH_SIZE` to 1 through 5,000 and defaults to 500. It repeats the batch operation up to `CIR_MAX_BATCHES_PER_RUN`, defaulting to 10, so a large backlog is drained incrementally rather than in one unbounded transaction.

Step 5: Build candidate predicates

The resolver ignores empty incoming values and creates an OR predicate for each populated active rule:

Rule family	Current SQL behavior	Example
exact scalar	<code>master_column = incoming_value</code>	email or phone
exact array identity	<code>incoming_value = ANY(master_array)</code>	device, advertising, or cookie ID
exact source-keyed ID	JSONB containment with the incoming <code>source_system</code> as key	<code>external_ids @> jsonb_build_object(source_system, id)</code>
exact domain attribute	EXISTS over the tenant and domain-matched <code>cdp_domain_profiles</code> row	banking national ID
fuzzy_trgm	<code>similarity(column, incoming_value) >= threshold</code>	address line or company name
fuzzy_dmetaphone	<code>dmetaphone(column) = dmetaphone(incoming_value)</code>	phonetic comparison when enabled

The resulting predicates are combined with OR, not AND. This means one trusted matching signal can be sufficient, while several signals increase the candidate score. The candidate query always constrains the master row by both `tenant_id` and `domain`.

The current active path is therefore a configurable hybrid matcher, not a fixed weighted identity model. Thresholds are metadata-driven, but the candidate score is currently the number of satisfied predicates divided by the number of predicates built for the incoming row. The resolver orders by this score and takes one candidate. There is no explicit deterministic tie-breaker after equal scores, so equal-score candidates remain an operational risk that should be addressed before high-stakes production use.

Step 6: Link and consolidate a match

When a candidate is found, the resolver inserts a row into `cdp_profile_links` and updates the existing master. The merge behavior is field-specific:

- `full_name`, `email`, and `phone_number` are scalar fields. Without consolidation metadata, the resolver uses COALESCE semantics and does not replace an existing value with a null value.
- Configured strategies include `non_null`, `overwrite`, `most_recent`, `verified_first`, `verified_then_most_recent`, `source_priority`, and `append_distinct`.
- `most_recent` compares an incoming timestamp field, then `event_time` or `created_at`, with the master timestamp.

- `verified_first` can use a configured verification field and values, or configured verified event names such as a KYC-completed event. If both sides have the same verification state, it uses its configured fallback.
- `source_priority` ranks the incoming source against the configured source list.
- Device, advertising, and cookie values are appended to their master arrays only when not already present.
- `external_customer_id` and `push_token` are written into source-keyed JSONB maps.
- `source_systems` is extended with a distinct source name.
- `communication_preferences` in the raw event payload are merged into the master JSONB document.
- `national_id` and `kyc_status` are consolidated in the domain profile's `domain_attributes` JSONB rather than in master scalar columns.

The resolver preserves the incoming domain on the master row. It does not merge two different domain masters merely because an identifier happens to be equal; the candidate query requires the same domain.

Step 7: Create a new master when no candidate exists

If no condition matches, `_create_master_and_link()` creates a `cdp_master_profiles` row. It initializes the source-keyed external ID map, identity arrays, push-token map, source-system list, and `first_seen_raw_profile_id`. It then inserts a `NewMaster` link and stores a national ID in the corresponding domain profile when present.

A raw row with no populated active matching attribute also follows the no-candidate path. This is intentional: the resolver does not invent an identity from empty data. It does mean that rule completeness and upstream normalization are important operational prerequisites.

Step 8: Recompute persona and finish the transaction

Persona resolution is enabled by default. After either a match or a new-master creation, `PersonaResolutionEngine.resolve_persona()` reads the resolved master and its domain attributes, computes a fresh result, and persists it. Only then does the resolver mark the raw row as processed and commit the transaction. Any resolver exception rolls back the transaction.

The persona engine catches its own exceptions and returns `None`, so a persona failure is logged without aborting otherwise successful identity matching work. This is a deliberate boundary: CIR establishes identity first; persona enrichment is downstream and non-blocking.

4. Persona Resolution After Identity Matching

Persona resolution is not an identity match and must not be used as evidence that two raw records represent the same person. It is an interpretation of one already-resolved master profile.

4.1 Scoring

The pure `compute_persona()` function calculates six component scores on a 0-100 scale:

- Behavior from lifecycle stage and existing engagement.
- Engagement from last-activity recency and source-system breadth.
- Financial value from predictive CLV, falling back to historical CLV.
- Loyalty from membership tier and customer tenure.
- Relationship from source-system breadth and secondary contacts.
- Risk from churn probability, risk segment, and KYC status.

The default positive weights are behavior 0.20, engagement 0.20, financial 0.20, loyalty 0.15, and relationship 0.10. The risk weight is 0.15 and is applied as $100 - \text{risk_score}$, so higher risk reduces the overall persona score. Thresholds, weights, and scoring constants are loaded from `cdp_persona_config` with in-code defaults and a short-lived cache.

The result includes a persona code and category, customer value tier, risk level, next-best action, component scores, confidence score, and explainability features. The current lookalike `match_score` is a confidence-score proxy; a dedicated embedding-distance calculation is not yet wired into the engine.

4.2 Persistence and history

The engine upserts one shared `cdp_persona_archetypes` row per `(tenant_id, domain, persona_code)`. It then inserts a versioned `cdp_customer_personas` assignment for the master profile, deactivates prior assignments, and updates `cdp_master_profiles.current_persona_id`, `persona_name`, and `persona_summary`.

The supporting tables preserve explainability:

- `cdp_persona_features` stores the signals used for one computation.
- `cdp_persona_score_details` stores component values, weights, formulas, and explanations.
- `cdp_persona_history` stores initial and material persona changes.

The relationship is many-to-many over time and across profiles: many master profiles can reference one shared archetype, while one master profile can receive many versioned assignments. The database trigger maintains each archetype's active matched-profile count.

The schema reserves `persona_embedding VECTOR(768)` on the shared archetype and an IVFFlat cosine index for future semantic or lookalike retrieval. The current persona computation does not generate or persist an embedding, so the paper treats this as available schema infrastructure rather than an active CIR matching step.

5. Execution and Concurrency

5.1 Near-real-time triggering

`IdentityResolutionTrigger.attempt_trigger()` uses the single-row `cdp_id_resolution_status` table as a throttle state. It tries to lock the row with `FOR UPDATE NOWAIT`:

1. If another worker owns the lock, it rolls back and returns `False` immediately.
2. If the row is missing, it logs the initialization problem and returns `False`.
3. If the last execution is less than `throttle_seconds` ago, it rolls back and defers the work.
4. Otherwise, it updates `last_executed_at` and runs one resolver batch.

The default throttle interval is five seconds. The controller catches errors so an ingestion request is not blocked by a resolution failure. The status table is a runtime support table initialized defensively by `backend-system/identity_resolution/scripts/init_sample_data.py` and represented in the API model.

5.2 Scheduled draining

`daily_job.py` provides the durable backlog path. It acquires a Redis lease under `identity-resolution:staging-dr`, opens PostgreSQL, executes bounded resolver batches, refreshes the lease after full batches, and releases the lease in `finally`. The Redis lease serializes complete staging drains across workers; the per-row PostgreSQL throttle remains the near-real-time coordination mechanism.

5.3 Transaction and retry behavior

One resolver invocation processes up to `batch_size` rows per tenant in a single database transaction. A successful invocation commits all changes. An exception rolls back the transaction. The processed status and unique raw-profile link prevent normal retries from repeatedly linking the same raw observation, while the pre-create link check handles a partially completed path.

6. Privacy and Tenant Safety

The demo and hashed-match ingestion path normalizes and SHA-256 hashes PII before persistence. `profile_looks_hashed()` identifies a populated `full_name`, `email`, `phone_number`, or `national_id` that matches the 64-character hexadecimal digest pattern. It sets `is_hashed` and creates a readable, non-PII persona label.

Persona naming has two fallback-safe paths:

1. If a real `LEO_GOOGLE_GENAI_API_KEY` is configured, Gemini receives only non-PII context such as domain and acquisition channel.
2. Otherwise, or when the SDK, network, key, quota, or response fails, an offline deterministic generator creates a domain role plus a stable six-character suffix.

The persona engine similarly sends only computed statistics to the optional LLM. It never passes raw profile PII to the LLM. The database check constraint ensures that a hashed master cannot be left without a display label.

Tenant safety has two layers:

- PostgreSQL RLS policies use `app.tenant_id` on master profiles, staging, links, domain profiles, identity indexes, persona tables, and other tenant-owned tables. The hardening migration enables and forces those policies and fails closed when the context is unset.
- Resolver SQL also filters by tenant. Candidate matching additionally filters by domain, and domain-attribute subqueries repeat tenant and master checks.

The runtime database role must not be a superuser or have `BYPASSRLS`; otherwise PostgreSQL will bypass the policy regardless of `FORCE ROW LEVEL SECURITY`.

7. Schema Capabilities Not Yet in the Active Batch Path

The database contains useful structures that should be distinguished from the current Python batch implementation:

- `cdp_identity_index` provides a unique tenant/type/normalized-identifier key and indexes for O(1)-style exact lookup. The current `CustomerIdentityResolver` still matches against master scalar fields, arrays, JSONB, and domain-profile JSONB; it does not populate or query this index in the active path.
- `cdp_profile_merge_history` stores source and target master snapshots for master-to-master merges and possible unmerge/split operations. The current raw-profile resolver does not write this table.
- `linked_raw_profile_count` and `last_identity_resolved_at` are schema/catalog fields intended for lineage and reporting. The current resolver links rows but does not update those denormalized fields in its shown batch logic.
- `address` is JSONB on the master, while the resolver's current scalar consolidation set is limited to `full_name`, `email`, and `phone_number`. The catalog seeds address matching rules, but the current raw-column projection omits those fields, and address consolidation into the master address document is not implemented by this resolver path.
- `first_name` and `last_name` are stored fields, but the seeded catalog does not enable them as identity keys.

These distinctions provide a practical roadmap: normalize and index active identifiers, implement deterministic tie-breaking, update lineage counters atomically, and add a controlled master-merge/unmerge workflow before treating those schema capabilities as production behavior.

8. Worked Example

Assume tenant T and domain retail contain this first observation:

```
raw A
  source_system      = Adjust
  email              = sha256(normalized customer email)
  device_id          = D1
  media_source       = TikTok Ads
```

No retail master in tenant T satisfies an active predicate, so CIR creates master M, stores the email, initializes `device_ids = [D1]`, records `source_systems = [Adjust]`, and inserts A → M with `match_method = NewMaster`.

Later, Web Tracking produces:

```
raw B
  source_system      = WebTracking
  email              = the same SHA-256 digest
  cookie_id          = C1
  event_name         = login
```

The resolver builds an exact email condition and an exact cookie condition. The cookie condition cannot match because C1 is not yet on M, but the email condition matches. The candidate score is $1 / 2 = 0.5$, and the method includes the field that matched. CIR inserts B → M, retains the existing master email, appends C1 to `cookie_ids`, adds WebTracking to `source_systems`, recomputes the persona for M, marks B as processed, and commits.

The example shows why match score and identity confidence are different concepts. The link score describes the conditions satisfied for this raw observation; the persona assignment's confidence score

describes the resolved master snapshot used by persona scoring. Neither score alone proves that two unrelated masters should be merged.

9. Evaluation and Operational Recommendations

The implementation is strong in the following areas:

- Explicit tenant and domain scoping at the candidate boundary.
- Metadata-driven matching and per-field consolidation.
- Preservation of raw lineage and cross-channel identifiers.
- Retry-aware links and transaction rollback.
- Optional, failure-contained persona enrichment.
- Privacy-aware hashed PII handling with an offline fallback.
- Redis serialization for scheduled backlog drains and PostgreSQL NOWAIT throttling for frequent triggers.

Before making the system a high-confidence production identity graph, the following controls should be completed or measured:

1. Add deterministic tie-breaking after candidate score ordering, preferably using identifier priority and stable master creation time.
2. Add calibrated, weighted match policies and review thresholds for fuzzy signals; an OR query with a single weak fuzzy match should not have the same operational meaning as an exact trusted identifier.
3. Populate and query `cdp_identity_index` for high-volume exact lookups, with normalization and blocked-value handling applied consistently.
4. Reconcile and maintain `linked_raw_profile_count` and `last_identity_resolved_at` as part of the same transaction as link creation.
5. Implement explicit failed-row handling, retry limits, and dead-letter monitoring if ingestion requires durable failure states.
6. Add a master-to-master merge, unmerge, and audit workflow using `cdp_profile_merge_history`.
7. Add integration tests for RLS with a non-superuser role, because a superuser connection bypasses RLS and cannot validate tenant isolation.
8. Measure precision, recall, false merges, false splits, backlog age, candidate latency, and persona enrichment latency separately.

10. Conclusion

The current Customer 360 solution merges multi-source profiles through a controlled raw-to-master process. A raw observation is scoped to its tenant and domain, compared with configured identity signals, linked to the best candidate when one exists, and consolidated without discarding source lineage. A non-match creates a new master rather than forcing an uncertain association. The result is then enriched by a separate, versioned persona engine.

This separation gives the platform a clear logical flow: staging preserves evidence, CIR makes the identity decision, master and link tables preserve the decision, and persona resolution adds explainable business interpretation. The schema already provides foundations for indexed identity lookup and reversible master merges, while the current implementation correctly documents those as follow-on capabilities rather than silently treating them as completed behavior.